

البرمجة النظيفة (Clean Code) تعتبر من أهم المفاهيم في تطوير البرمجيات، لأنها تساعد على كتابة أكواد واضحة وسهلة الفهم والتعديل. عندما يكون الكود منظماً، يصبح العمل على المشروع أسهل سواء للمبرمج نفسه أو لفريق العمل. في هذا التقرير سيتم توضيح أربعة مفاهيم أساسية:

Clean Code

Project Structure

Namings

DRY

أولاً: Clean Code

ما هي؟

البرمجة النظيفة هي طريقة لكتابة الأكواد بشكل مرتب وواضح بحيث تكون سهلة القراءة والفهم والصيانة.

لماذا توجد؟

توجد لحل مشاكل الفوضى البرمجية وصعوبة فهم الكود، خاصة في المشاريع الكبيرة. كما تساعد على تقليل الأخطاء وتسهيل تطوير المشروع مستقبلاً.

أمثلة على فائدتها

مثال غير واضح:

```
int x = 10;
```

مثال أوضح:

```
int studentCount = 10;
```

الاسم الثاني يوضح وظيفة المتغير بشكل أفضل.

ثانيًا: Project Structure

ما هو؟

Project Structure يعني تنظيم ملفات ومجلدات المشروع بطريقة مرتبة حسب وظيفة كل جزء.

لماذا يوجد؟

يساعد على سهولة الوصول للملفات وتنظيم المشروع وتقليل الفوضى، خاصة عند العمل ضمن فريق.

مثال

lib/ |— screens/ |— models/ |— services/ |— widgets/
كل مجلد يحتوي على نوع معين من الملفات، مما يجعل المشروع أسهل في الإدارة.

ثالثًا: Namings

ما المقصود بها؟

Namings تعني اختيار أسماء واضحة للمتغيرات والدوال والملفات.

لماذا توجد؟

لأن الأسماء الجيدة تجعل الكود سهل الفهم، بينما الأسماء السيئة تسبب صعوبة في القراءة والتعديل.

مثال

أسماء غير واضحة:

String a;

أسماء واضحة:

String studentName;

الاسم الواضح يساعد المبرمج على فهم الكود بسرعة.

رابعًا: DRY – Don't Repeat Yourself

ما هو؟

هو مبدأ يعني عدم تكرار نفس الكود أكثر من مرة داخل المشروع.

لماذا يوجد؟

لأنه يقلل حجم الكود ويسهل تعديله ويقلل الأخطاء.

مثال

كود مكرر:

```
print("Hello"); print("Hello");
```

باستخدام دالة:

```
void showMessage() { print("Hello"); }
```

إعادة استخدام الدالة أفضل من تكرار الكود.

الخاتمة

تساعد مبادئ Clean Code على كتابة برامج منظمة وسهلة التطوير والصيانة. كما أن تنظيم المشروع، اختيار الأسماء المناسبة، وتجنب تكرار الكود يجعل العمل البرمجي أكثر احترافية ويوفر الوقت والجهد.

